Reinforcement Learning and Central Pattern Generator on Quadruped Robot Control

1st Boshi An
Mircoengineering Section
EPFL
Lausanne, Switzerland
boshi.an@epfl.ch

2nd Di Xiao
Mircoengineering Section
EPFL
Lausanne, Switzerland
di.xiao@epfl.ch

3rd Zhuoran Ren
Mircoengineering Section
EPFL
Lausanne, Switzerland
zhuoran.ren@epfl.ch

***Abstract*—In this article, we demonstrated our project results for the course Legged Robots 2024 Fall. We mainly compared different methods (Central Pattern Generator (CPG), Reinforcement Learning (RL), CPG+RL on different tasks, forward run, stair climbing, etc.). We also did some modifications to the original simulation settings, and provided ablation study to quantify the contributions of all the modifications.**

I. Introduction

Modeling locomotion is a fundamental task in robotics, with three primary approaches commonly employed: model-based, learning-based, or bio-inspired methods. This study focuses on the latter two approaches. First, we utilize central pattern generators (CPGs) to implement various gaits for quadrupedal locomotion. Next, we design a Markov Decision Process (MDP) framework to learn these same types of gait patterns through deep reinforcement learning (DRL) algorithms. We also put CPG and RL methods together to form a CPG-RL controller. Finally, we conduct a comparative analysis to evaluate the performance and characteristics of all the methods.

We have included videos of all the methods we investigated in the submission. You can find them in the folder `saves`.

II. Tasks

Forward Locomotion: The robot needs to run as fast as possible without falling on a flat surface.

Stairs: The robot needs to climb n stairs without falling.

Gap: The robot needs to jump across n gaps without falling.

Random Block: The robot needs to move through a space fall of random blocks.

All: One of the terrains are assigned to the robot to perform locomotion.

III. Methods

A. Central Pattern Generator (CPG)

$$\dot{r}_i = \alpha(\mu - r_i^2)r_i \tag{1}$$

$$\dot{\theta}_i = \omega_i + \sum_{j=0}^{3} r_j w_{ij} \sin(\theta_j - \theta_i - \phi_{ij}) \tag{2}$$

$$x_{\text{foot}} = -d_{step} r_i \cos(\theta_i) \tag{3}$$

$$z_{\text{foot}} = \begin{cases} -h + g_c \sin(\theta_i) & \text{if } \sin(\theta_i) > 0, \\ -h + g_p \sin(\theta_i) & \text{otherwise.} \end{cases} \tag{4}$$

In equations 1 and 2, r_i is the amplitude of the oscillator, α is the positive speed convergence constant, which controls how fast the oscillator will reach to the desired amplitude μ. θ is the phase of the oscillator, influenced by the intrinsic frequency ω_i and the coupling strength between the oscillator itself and other oscillators ω_j. ϕ_{ij} is the desired phase offset between the oscillators i and j, which depends on the different gaits.

Equations 3 and 4 convert the states into Cartesian foot positions in xz coordinate, where d_{step} is the step length, h is the robot height, g_c and g_p are the maximum ground clearance and penetration corresponding to the swing or stance cycle.

B. Reinforcement Learning (RL) and CPG-RL

1) Environment Design: We have done some changes to the original environment for RL methods.

- More Domain Randomization: Apart from changing the mass distribution of the robot at the beginning of each episode, we added randomization to the initial pose and joint position to the robot.
- Random Tasks: In all the tasks except `Forward Locomotion`, we added randomization to the size and separation of each obstacle. In the `Stairs` task, the number of stairs, length of stairs and the height of stairs all can vary. In the `Gaps` task, the separation of gaps and the gap length are randomized. In the `Slopes` task, the inclination and length of the slope can change. In the `Random` task, the height of each object are different.
- Lidar Scan Dots: We implemented a lidar for the robot. The robot can see the height-map around it.

2) Observation Space: We have added some other observations for the agent.

- Joint angles
- Joint velocities
- Joint torques

- y and z
- Base orientation in quaternion
- linear velocity
- Base angular velocity
- yaw pitch and roll rate
- relative position
- Foot relative velocity
- action
- in contact
- Foot normal forces
- Foot friction forces
- in flight accumulated time
- in contact accumulated time
- Lidar scan dot distance (optional)

For CPG-RL, we also add vectors $r, \theta, \dot{r}, \dot{\theta}$ for the observation.

3) Action Space: For different control modes, we have different action spaces.

The basic control mode is torque control, where the action is directly the torques to be scaled and applied to the motors. This control mode has an action space dimension of 12.

The action space for joint PD is the desired position of each joint. This control mode has an action space dimension of 12.

The action space for Cartesian PD is the desired relative 3d position of each foot. This control mode has an action space dimension of 12.

For CPG-RL, the policy outputs α, μ, ω, y. Here α, μ, ω are described in the section III-A , y is the desired foot y, which is not provided by the CPG oscillator. This control mode has an action space dimension of 14.

4) Reward Function: We have tested and chosen several reward terms to form the final reward function, insuring stable and efficient training.

$$\begin{aligned}
\text{vel_tracking_reward} &= \text{clip}(\text{sign}(v_x)\sqrt{v_x}, -1, 1) \\
\text{foot_switch_reward} &= \sum_{i=1}^{4} (\sqrt{\text{cc}_i} - -0.065\text{cc}_i) \times [\text{cc}_i > 5] \\
&\quad + \sum_{i=1}^{4} \left(\sqrt{\text{cf}_i} - 0.065\text{cf}_i\right) \times [\text{cf}_i > 5] \\
\text{foot_lift_reward} &= \sum_{i=1}^{4} (\text{foot_z}_i - 0.1)\sqrt{\|\text{foot_v}_i\|_2} \\
\text{foot_slip_reward} &= -\sum_{i=1}^{4} \|\text{foot_v}_i\|_2^2 \times \text{foot_cont}_i \\
\text{up_reward} &= \frac{1}{\|\text{z} - [0\ 0\ 1]^T\|_2^2 + 0.4^2} \\
\text{yaw_reward} &= -\phi^2 \\
\text{yaw_correction_reward} &= \text{sign}(\phi)\text{sign}(\dot{\phi})
\end{aligned}$$

$$\begin{aligned}
\text{init_joint_reward} &= -\sum_{i=1}^{12} (q_i - q_{\text{ini},i})^2 \\
\text{drift_reward} &= -y^2 \\
\text{drift_correction_reward} &= \text{sign}(y)\text{sign}(\dot{y}) \\
\text{fall_reward} &= -\text{is_fall} \\
\text{success_reward} &= \text{is_success} \\
\text{out_reward} &= \text{is_out} \\
\text{energy_reward} &= \sum_{i=1}^{12} \tau_i \dot{q}_i \\
\text{steady_reward} &= -[\dot{x} < 0.1]
\end{aligned}$$

Where cc_i is the accumulated step count for continuous time-steps i-th foot has contact on the ground, while cf_i is the accumulated step count for continuous time-steps i-th foot is in the fly. We carefully adjusted the weights for each reward term, all the coefficients can be found in our code.

The effect of each of the reward terms can be inferred by their names. To make it clearer, here are the explanations of some of the terms which are difficult to comprehend:

- The foot_switch reward creates a convex function, where if the agent steps with a specific frequency, it will gain the maximum reward. This reward term is to prevent the agent from vibrating its feet to gain speed.
- The foot_lift_reward encourages the agent to lift its foot and swing it fast.

5) Algorithm: We used Proximal Policy Gradient [1] (PPO) and Soft Actor Critic [2] (SAC).

For the policies, we used a default network of 256×256 hidden size Multi-layer Perceptron (MLP).

6) Curriculum: For the ease of training, we adopted the broadly-used curriculum learning methods. When the success rate on a specific environment achieves a pre-defined level, the algorithm will automatically increase the difficulty of the environment.

IV. Experiments

A. CPG

1) Gait Plotting: We plotted the states r, $\dot{r}$ and $\dot{\theta}$ of all legs under the trot gait.

The meaning of each state is detailed in eq. (1) and eq. (2). On the left side of the figure, the sharp peaks in $\dot{r}$ correspond to moments when r increases, eventually stabilizing at its desired amplitude as $\dot{r}$ approaches 0. On the right side, the phase (θ) of the front left leg aligns with that of the back right leg, while the front right leg is in phase with the back left leg. This pattern accurately represents the trot gait commonly observed in quadrupedal animals.

2) Cartesian Position controller: We used Cartesian Position Controller, Joint PD Controller and the combination of these two controllers as the low-level controller of CPG, and plotted their performance:

To analyze the effect of different control strategies on trajectory following during the trot gait, we focusing solely

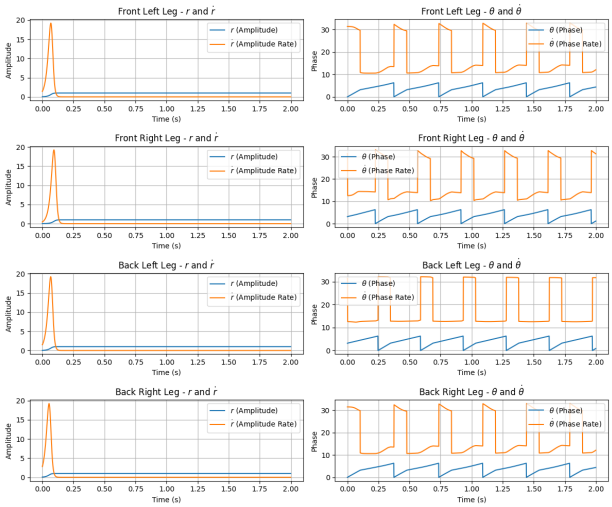

Fig. 1: CPG states (r, θ, $\dot{r}$, $\dot{\theta}$) for the trot gait

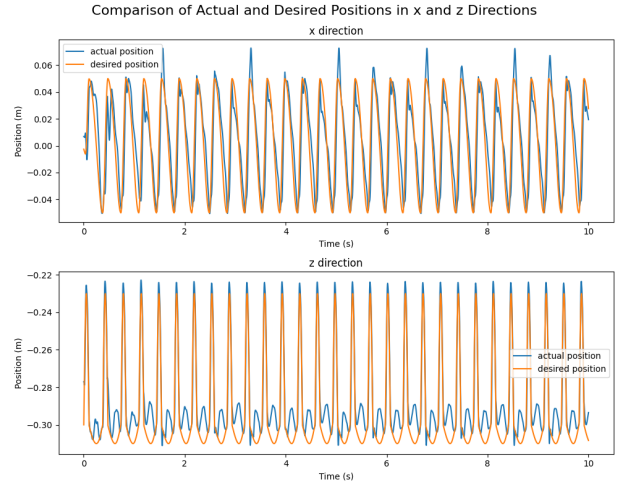

Fig. 2: Trot gait: desired FR foot position vs. actual FR foot position with Cartesian PD

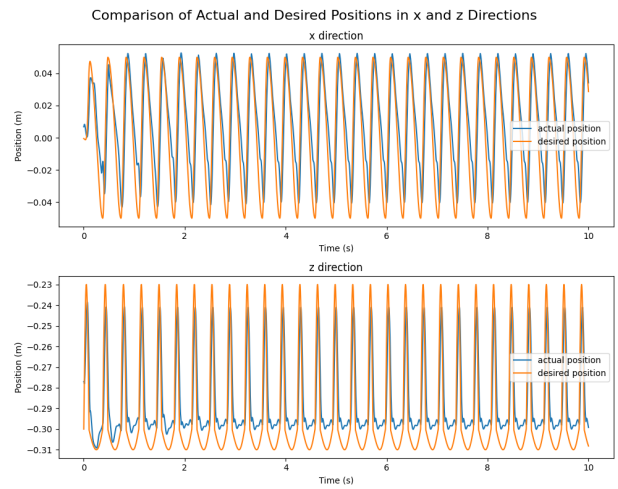

Fig. 3: Trot gait: desired FR foot position vs. actual FR foot position with joint PD

on the tracking accuracy of the front-right leg (FR). We used the following gain matrices.

For Cartesian PD:

$$K_p = \begin{bmatrix} 1000 & 0 & 0 \\ 0 & 1000 & 0 \\ 0 & 0 & 4000 \end{bmatrix} \tag{5}$$

$$K_d = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 10 \end{bmatrix} \tag{6}$$

For joint PD:

$$K_p = \begin{bmatrix} 100 & 100 & 100 \end{bmatrix}, \quad K_d = \begin{bmatrix} 2 & 2 & 2 \end{bmatrix} \tag{7}$$

The Joint PD controller operates directly in the joint space, precisely adjusting each joint angle with minimal dependence on the kinematic chain. However, it lacks an explicit mechanism to ensure precise end-effector positioning in task space. In contrast, the Cartesian PD controller regulates the end-effector in task space, relying on inverse kinematics for transformations and managing coupling between joints. This reliance makes it susceptible to inaccuracies in the kinematic model and unmodeled dynamics, leading to discrepancies and reduced performance.

A comparison of the results in fig. 2 and fig. 3 indicates that both the Cartesian PD Controller and Joint PD Controller struggled to accurately track the desired foot position. The Cartesian PD controller exhibited more frequent and significant overshoots, particularly in the horizontal (x) direction, likely due to modeling inaccuracies and dynamic coupling effects. While the joint PD controller performed better in the horizontal direction, it also showed tracking offsets in the vertical (z) direction, as it struggles to account for cumulative effects in task-space trajectories.

Combining joint and Cartesian control in fig. 4 significantly improved performance. This approach reduced the vertical (z) offset and achieving closer tracking of the desired position during stabilization by leveraging the joint controller's precision and the Cartesian controller's global trajectory management.

3) Joint Position Controller: Similarly, for the joint position control, the Cartesian PD, joint PD, and both are applied to the three joints of the front right (FR) leg. The gain matrix used is the same as in section IV-A2.

The Cartesian PD controller in fig. 5 did not perform well, displaying significant overshoots in all three joints. This outcome is expected, as the Cartesian controller operates in task space and relies on inverse kinematics, making it prone to modeling inaccuracies and dynamic coupling effects. In contrast, the joint PD controller in fig. 6 tracked the thigh joint accurately but exhibited deviations in the hip and calf joints, likely due to its inability to fully address dynamics couplings between joints and the use of a suboptimal PD gain matrix.

By combining both joint PD and Cartesian PD control in

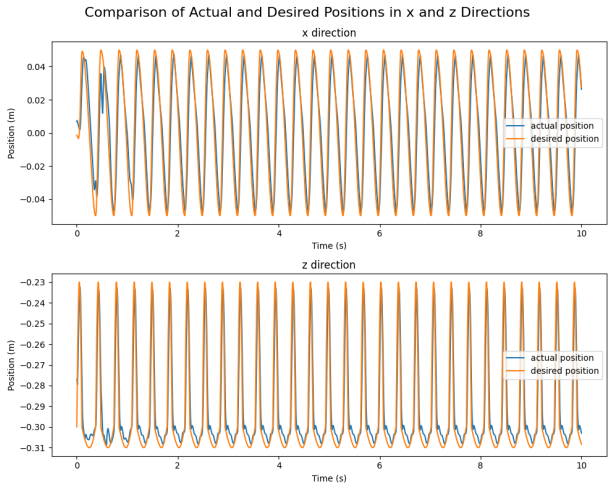

Fig. 4: Trot gait: desired FR foot position vs. actual FR foot position with both joint and Cartesian PD

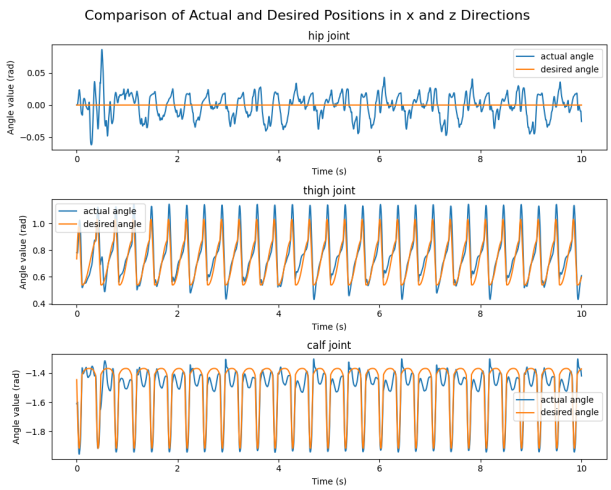

Fig. 5: Trot gait: desired FR joint position vs. actual FR joint position with Cartesian PD

fig. 7, performance improved slightly compared to using the joint controller alone. The actual joint angles more closely matched the desired angles for the thigh and calf joints. Furthermore, the oscillations in the hip joint became better centered around the reference angle of 0 degrees.

4) Parameters Tuning: To investigate the impact of different parameters on CPG, a grid search method was employed. Three parameters were selected due to their significant influence on the behavior of the robot in trot gait: ω_{swing}, μ and the coupling strength ω_{ij}. In addition, we set the duty factor to 0.4. These parameters are discussed in detail below:

- ω_{swing}:This parameter will significantly influence the average speed of the robot. We limit this value to be within $[15\pi, 40\pi]$ in the grid search.
- μ: eq. (2) ensures that r_i converges to $\sqrt{\mu}$. Since r_i will influence the robot speed directly according to eq. (3), we choose to tune μ in our grid search and search its value from 0.1 to 1.
- ω_{ij}: The coupling strength ω_{ij} will affect the frequency of swings of the feet and the robustness of the gait. We are tuning this value between 0.1 and 3.

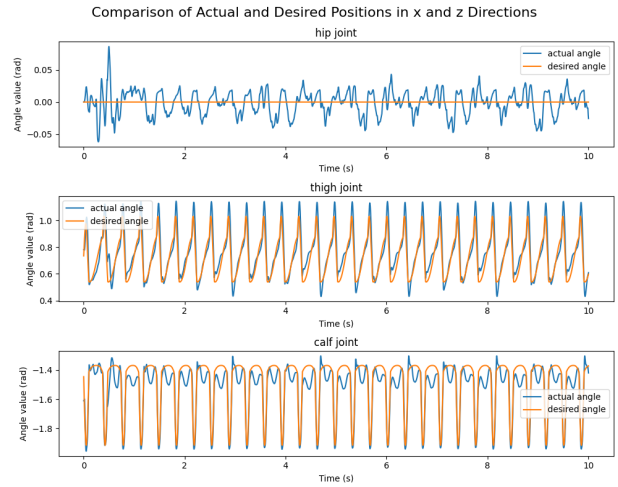

Fig. 6: Trot gait: desired FR joint position vs. actual FR joint position with Cartesian PD

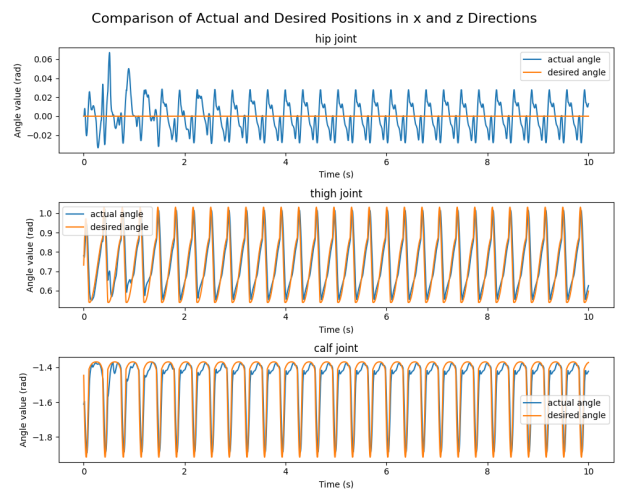

Fig. 7: Trot gait: desired FR joint position vs. actual FR joint position with both joint and Cartesian PD

The result of the grid search is shown by the fig. 8, fig. 9 and fig. 10:

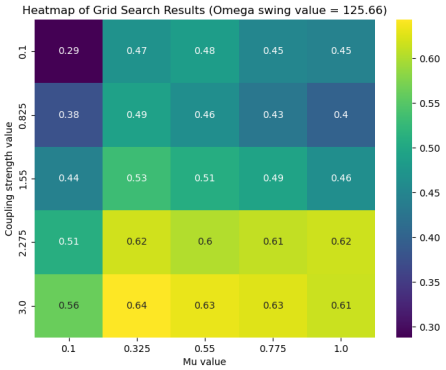

Fig. 8: Grid search result when fixing ω_{swing} to 40π

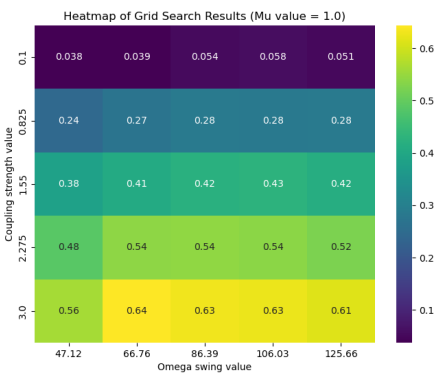

Fig. 9: Grid search result when fixing μ to 1.0

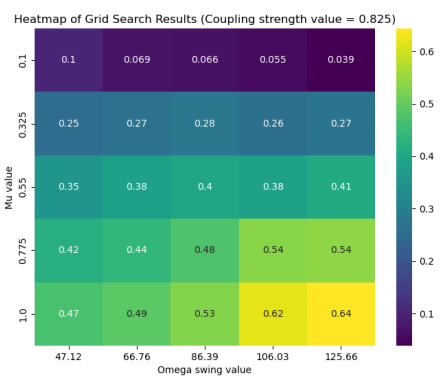

Fig. 10: Grid search result when fixing ω_{ij} to 0.825

To conclude, the minimal and maximal velocity we have achieved for the quadruped robot in trot gait is 0.038 m/s and 0.64 m/s respectively, and the parameters setting for this result can been shown in the table I and table II.

5) Velocity Tracking: The following implementation demonstrates a CPG controller designed to track a piecewise velocity profile. It uses a feedback loop to address the error between the desired velocity and the actual detected velocity.

Parameter name	Value
average speed	$0.038m/s$
duty ratio	71.43%
swing duration	$0.133s$
stance duration	$0.333s$
energy cost	$788.7J$
Cost of Transport	3.833

TABLE I: parameters setting for slowest trot gait performance

Parameter name	Value
average speed	0.64 m/s
duty ratio	71.43%
swing duration	$0.050s$
stance duration	$0.125s$
energy cost	$3077J$
Cost of Transport	4.21

TABLE II: parameters setting for fastest trot gait performance

In this case, only proportional control is used, with a proportional gain K_p = 35.

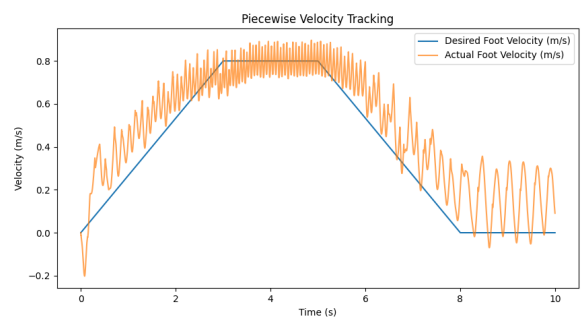

Fig. 11: Change of speeds

6) Discussion: The CPG combined with a joint/Cartesian PD controller represents an open-loop model that is tuned for a specific flat environment and can only handle small perturbations. To enhance robustness, sensory feedback can be integrated into the oscillators, enabling the system to account for disturbances and execute rapid corrections. For instance, a stumbling reflex can be implemented to adjust foot placement during the swing phase, helping the robot recover from collisions with obstacles and reducing the risk of falls. In addition, posture control feedback mechanisms (including attitude control, attack angle control, direction control) can continuously adjust the robot's body orientation and position, facilitating stable locomotion over uneven or unknown rough terrain.

B. Comparison and Ablation Study

We conducted an ablation study on different parts of the model.

1) Control Methods: We compared the performance of four different methods: `Torque`, `Joint PD`, `Cartesian PD` and `CPG RL`. We compared all the methods in the forward locomotion task, where the agent is required to move forward as fast as possible.

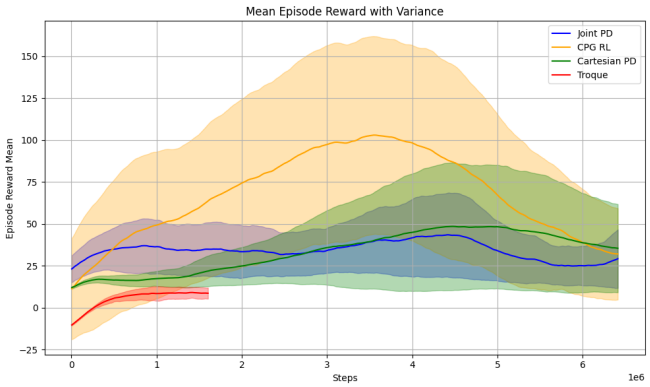

Fig. 12: Comparisons on different control methods

From fig. 12, we can notice that `CPG RL` has outperformed all the other methods with a large margin. Among the other methods, `Cartesian PD` and `Joint PD` has similar performance, while `Torque` is struggling to achieve higher reward.

The shaded area in the plot represents the standard deviation of the reward in different runs.

2) Networks: We compared the performance of `default MLP`, `large MLP`, `LSTM` used in CPG-RL setting, on forward locomotion task. The `default MLP` has a fully-connected network of hidden size 256 and 2 hidden layers. The `large MLP` has a fully-connected network of hidden size 512 and 2 hidden layers. The `LSTM` network uses a LSTM (Long-short Term Memory network) of hidden size 256 and 2 hidden layers.

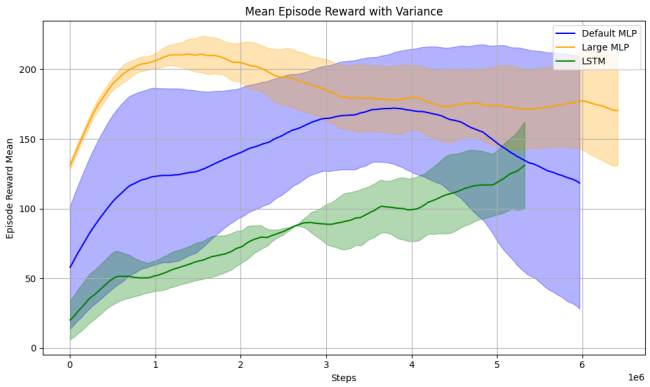

Fig. 13: Comparisons on different control methods

From fig. 13, we can infer that larger MLP networks can benefit the training, while LSTM is not improving the performance. We can also notice that they all have a performance decrease close to the end of the training. We think this is due to problems in the reward design leading it to strange behaviors.

3) Training Algorithm: We compared the performance of `PPO` and `SAC` in forward locomotion task.

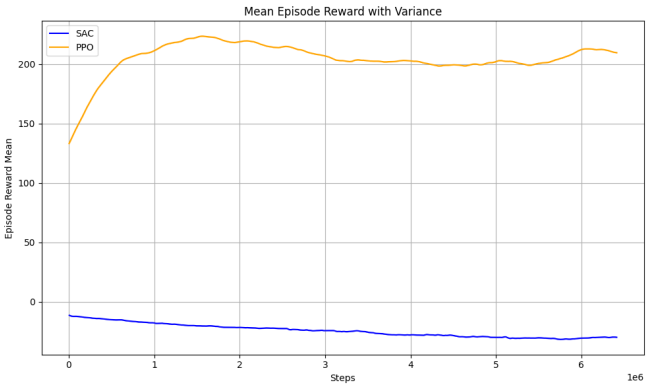

Fig. 14: Comparisons on different control methods

From fig. 14, we can see that PPO is better than SAC with the given hyper-parameter setting.

4) Lidar Observation: We compared the performance with and without `Lidar` Observation and with `LSTM` when co-training on all tasks (Stairs, Slope, Gap, Random). We have assumed that the co-training of all tasks requires at least some exteroceptive abilities, or memory abilities of the agent.

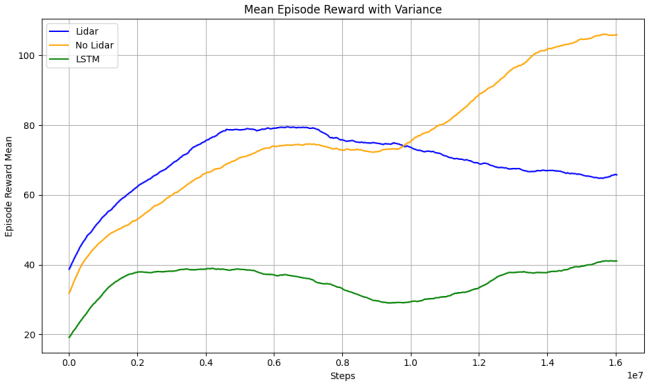

Fig. 15: Comparisons on Lidar observation and LSTM

From fig. 15, we can observe that Lidar observation is in fact not helping the model, and memory abilities provided from the LSTM network is also not improving the performance, which is contradictory with our assumption. This may be due to the Lidar observation and the LSTM network are not tuned properly, and only the proprioceptive function is enough for the task it is trained on.

5) Curriculum Learning: We compared the performance with and without `Curriculum Learning` on the Stairs setup.

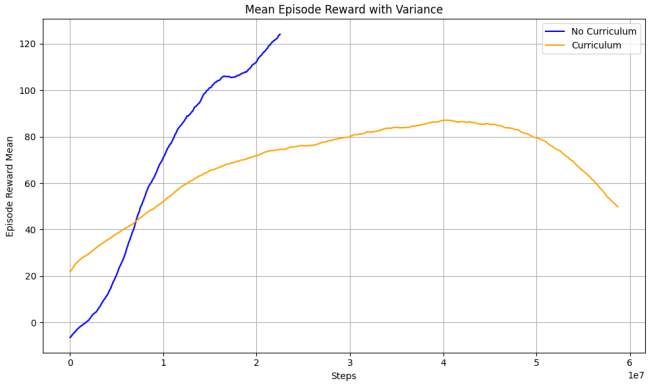

Fig. 16: Comparisons on Curriculum Learning and Direct RL

The results in fig. 16 are interesting. Curriculum Learning makes performance worse. However, when we tested the overall success rate in the fully randomized environment (see section III-B1 for details), the model trained with Curriculum Learning achieved success rate (probability of reaching $x = 5.0$) of **0.47**, while the model directly trained in the fully randomized environment with RL has success rate of **0.44**.

ACKNOWLEDGMENT

REFERENCES

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International conference on machine learning*. PMLR, 2018, pp. 1861–1870.